Unidade 6 — Sistemas multiagentes

Aplicações de Modelos de Linguagem Natural

O que é um sistema multiagente

- Até aqui, vimos o agente isolado: uma entidade que planeja, executa e reflete sozinha.
- Um **sistema multiagente** (*multi-agent system*, **MAS**) contém vários agentes que **colaboram, se coordenam ou até competem** para atingir um objetivo.
- Cada agente tem algum grau de autonomia, mas pode ter ferramentas, subobjetivos ou até personalidades diferentes.

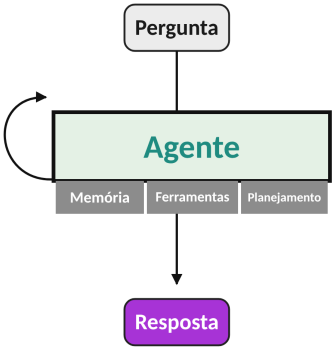
Exemplo: um sistema de pesquisa

Em vez de um único agente fazer tudo (buscar na web, processar artigos e figuras, ler e criticar os artigos, resumir), teríamos um **agente de busca**, um **de processamento**, um **de leitura de artigos**, um **de sumarização**, e assim por diante.

Agente único × sistema multiagente

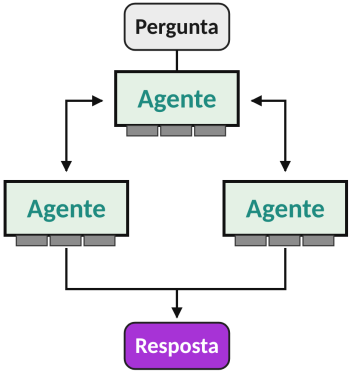
Agente único

(itera sobre o próprio comportamento)



Multiagente

(itera sobre o comportamento uns dos outros)



Ambientes e papéis

- Os agentes podem ou não compartilhar o mesmo ambiente. Por exemplo, um **sistema de reservas** com um agente que procura lugares para visitar, outro que pesquisa voos baratos e outro que organiza a agenda das férias.
- Os ambientes podem ser:
  - **físicos** (robôs);
  - **virtuais** (ambientes de jogo);
  - **textuais** (IDEs de programação, interfaces de chat).
- Quanto aos papéis, os agentes podem ser:
  - **homogêneos**: capacidades parecidas, para executar tarefas em paralelo;
  - **heterogêneos**: papéis e objetivos diferentes, possivelmente com LLMs, memórias e ferramentas diferentes;
  - com **especialização emergente**: agentes idênticos que, interagindo, evoluem para papéis especializados, como coletar recursos ou defender, em simulações de jogos.

Vantagens, desvantagens e uso na prática

Vantagens

Colaboração → problemas complexos e diversos ficam mais fáceis com mais agentes trabalhando juntos.

Escalabilidade → é possível aumentar o número de agentes sem deixar o sistema mais lento, rodando-os em paralelo ou de forma assíncrona.

Velocidade → vários agentes resolvem tarefas em paralelo, e agentes especializados tendem a ser mais rápidos que um agente genérico.

Desvantagens

Complexidade → é preciso gerenciar com cuidado a orquestração e as interações entre agentes.

Custo → mais agentes com LLMs poderosos significam mais computação.

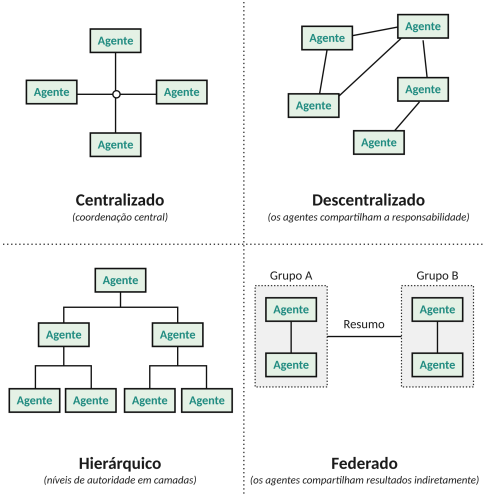
Avaliação → avaliar um agente já é difícil; avaliar vários trabalhando juntos é ainda mais.

Padrões de orquestração

Colocar vários agentes em um sistema exige decidir como eles serão organizados, com quais especialidades e como vão se comunicar. Os **padrões** descrevem a topologia do sistema, ou seja, como os agentes se relacionam e se comunicam. São quatro tipos:

- **Centralizado:** um único agente orquestrador controla a comunicação e distribui responsabilidades e tarefas. É o mais usado, por ser fácil de implementar e gerenciar; a desvantagem é que o sistema todo pode cair se o orquestrador falhar.
- **Descentralizado:** os agentes têm responsabilidades parecidas e a comunicação é distribuída, sem um "líder". É o mais estável (se alguns agentes falham, os outros continuam), e escalar é só acrescentar agentes; mas exige muitos agentes capazes e muita comunicação, o que consome recursos.
- **Hierárquico:** os agentes ficam em camadas, cada nível com um grau de autoridade e papéis distintos. É intuitivo, pois lembra as organizações tradicionais, e permite alocar recursos de forma eficiente; ainda assim, é complexo e é o que exige mais comunicação.
- **Federado:** os agentes ficam em ambientes paralelos (organizações diferentes) e se comunicam apenas indiretamente, para proteger a privacidade dos dados; cada grupo segue regras diferentes, mas pode compartilhar resultados. É um padrão recente, que favorece a colaboração entre sistemas, mas depende de padrões compartilhados que nem sempre existem.

5.2 Os quatro padrões de orquestração



Subagentes como ferramentas

Um "truque" simples para criar um sistema centralizado (e potencialmente hierárquico) é tratar os **subagentes como ferramentas**. Cria-se um **agente de matemática** (ferramentas somar, subtrair e multiplicar) e um **agente de datas** (ferramentas "data de hoje" e "dias entre duas datas"). Cada subagente é "embrulhado" em uma função (`ask_math_agent` e `ask_date_agent`), entregue como ferramenta a um **agente orquestrador**, que decide quando pedir ajuda a cada um.

Trajetória do exemplo

Pergunta ao orquestrador: "Se eu economizar €4 por dia até 2030, quanto terei?"

Passo 1 → ferramenta `ask_date_agent`("Quantos dias de hoje até o fim de 2030?")
Observação: "São 1.691 dias de hoje (15 de maio de 2026) até o fim de 2030 (31 de dezembro de 2030)."

Passo 2 → ferramenta `ask_math_agent`("€4 vezes 1691")
Observação: "€6.764"

Passo 3 → resposta: "Se você economizar €4 por dia até o fim de 2030, terá €6.764."

Trajetória interna do agente de datas:

Passo 1 → ferramenta `today()` → 2026-05-15

Passo 2 → ferramenta `days_between("2026-05-15", "2030-12-31")` → 1691

Passo 3 → resposta: "São 1.691 dias..."

Subagentes como ferramentas: duas lições

Esse exemplo traz duas lições:

1. A ideia é útil quando há **dezenas ou centenas de ferramentas**: o orquestrador erraria ao escolher entre tantas opções e pode, em vez disso, dedicar subconjuntos delas a subagentes.
2. Olhar só a trajetória do orquestrador dá uma **visão abstrata** do que aconteceu; por isso, é preciso inspecionar também as **trajetórias dos subagentes** para saber se eles seguiram um caminho eficiente.

5.4 Frameworks para sistemas multiagentes: CAMEL e MetaGPT

Não existe um único framework ou padrão ideal para todos os casos; cada framework tem a sua "filosofia".

- **CAMEL:** um dos primeiros frameworks colaborativos, baseado em **interpretação de papéis** (*role-playing*). O usuário propõe uma ideia (por exemplo, "criar um site para o meu blog de IA") e são criados dois agentes: o **usuário de IA**, que representa o usuário e dá instruções (aqui no papel de "blogueiro"), e o **assistente de IA**, que executa (no papel de "programador"). Um **agente especificador de tarefas** reescreve o pedido de forma mais detalhada (por exemplo, "crie um site pronto para produção, usando frameworks modernos como Next.js e Tailwind CSS..."). Em seguida, os dois agentes conversam em vários turnos até o usuário de IA encerrar e devolver a resposta.
- **MetaGPT:** leva a interpretação de papéis mais longe e imita **estruturas organizacionais inteiras**, como uma empresa de software. Tem três componentes:
  - **especialização de papéis**: cada papel tem nome, objetivo, habilidades e restrições, como gerente de produto, arquiteto e engenheiro, e todos seguem um procedimento operacional padrão, trabalhando em sequência, cada um com ReAct;
  - **protocolo de comunicação**: os agentes se comunicam por artefatos estruturados, como documentos e diagramas, e não só por diálogo;
  - **programação iterativa com feedback executável**: cada agente revisa a própria saída; o engenheiro, por exemplo, também executa e depura o código.

5.4 Frameworks para sistemas multiagentes: frameworks de produção

- **Frameworks de produção:** criados do ponto de vista do desenvolvedor, como produtos para construir e escalar sistemas multiagentes.
- No fim de 2025, os mais comuns eram o **AutoGen** (Microsoft), o **LangGraph** e o **CrewAI**, todos modulares, permitindo escolher papéis, memória, ferramentas e o padrão do sistema.
- O **LangGraph** será usado na parte prática da **Unidade 7**.
- Também existe o **n8n**, ferramenta de código aberto de automação de fluxos com IA, em que os fluxos são predefinidos, mas os agentes ainda podem ter alguma autonomia ("no limite" de um sistema multiagente).